Promises in JavaScript

Asynchronous Programming • resolve • reject • then • catch • async/await

1. What is a Promise?

A Promise is a JavaScript object that represents the future result of an asynchronous operation.

It is useful when the result is not available immediately.

Common examples: API requests, database operations, timers, file operations, and server communication.

A Promise does NOT make JavaScript block the entire program.



2. Promise States

PENDING
Operation is still running

FULFILLED
Operation completed successfully
→ resolve()

REJECTED
Operation failed
→ reject()

A Promise starts in the pending state.

It can settle only once: fulfilled OR rejected.

After it settles, the state cannot change again.

3. Why do we need Promises?

Asynchronous work may take an unknown amount of time.

JavaScript should continue executing other code instead of freezing while waiting.

Promises provide a structured way to receive the result later.

They also provide a standard way to handle success and failure.

```
console.log("1");

setTimeout(() => {
    console.log("Data received");
}, 2000);

console.log("3");
```

Output: 1 → 3 → Data received

4. What can go wrong without proper handling?

```
let data;

setTimeout(() => {
    data = "Student Data";
}, 2000);

console.log(data);
```

Output: undefined

The asynchronous operation starts.

JavaScript continues immediately.

console.log(data) runs before the data is ready.

Two seconds later, data is assigned—but the earlier console.log already happened.

5. Creating a Promise

```
function getStudentData() {
    return new Promise((resolve, reject) => {

        const success = true;

        if (success) {
            resolve({
                id: 101,
                name: "Rahim",
                department: "CSE",
                cgpa: 3.92
            });
        } else {
            reject("Failed to get student data");
        }

    });
}
```

new Promise((resolve, reject) => { ... }) creates a Promise.

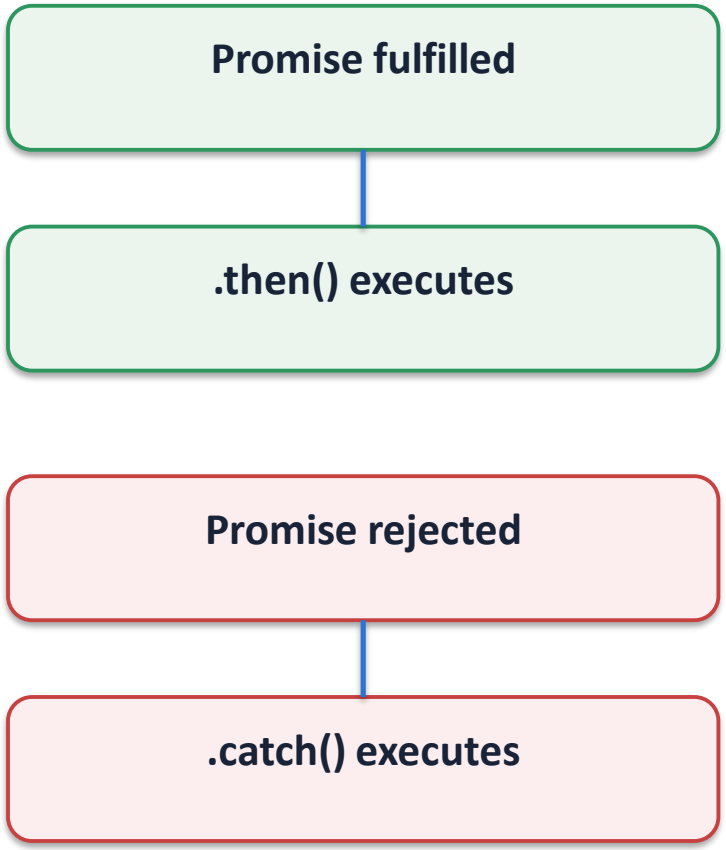
resolve(value) means success.

reject(error) means failure.

The Promise returned by the function can be handled later.

6. Handling a Promise with .then() and .catch()

```
getStudentData()
    .then((student) => {
        console.log("Student data received!");
        console.log(student.name);
    })
    .catch((error) => {
        console.log("Error:", error);
    });
```



7. Complete Example: Student Data

```
function getStudentData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = true;

      if (success) {
        resolve({
          id: 101,
          name: "Rahim",
          department: "CSE",
          cgpa: 3.92
        });
      } else {
        reject("Failed");
      }
    }, 2000);
  });
}

getStudentData()
  .then(student => console.log(student))
  .catch(error => console.log(error));
```

The Promise starts as PENDING.

setTimeout simulates a server/API delay.

After 2 seconds, resolve() fulfills the Promise.

.then() receives the student object.

If reject() runs, .catch() handles the error.

8. Modern Approach: async / await

```
async function displayStudent() {
    try {
        console.log("Getting student data...");

        const student = await getStudentData();

        console.log("Student data received!");
        console.log("ID:", student.id);
        console.log("Name:", student.name);
        console.log("CGPA:", student.cgpa);

    } catch (error) {
        console.log("Error:", error);
    }
}

displayStudent();
```

async makes a function asynchronous.

await waits for the Promise result inside that async function.

try/catch handles success and failure.

async/await is often easier to read than long .then() chains.

await does NOT block the entire JavaScript application.

9. The Complete Flow



Example: API request → server processes request → response arrives → Promise resolves → UI uses the data.

This same concept appears in fetch(), databases, Node.js, Express, and many modern JavaScript libraries.

10. Promise vs Callback

Callbacks can handle asynchronous operations, but deeply nested callbacks can become difficult to read and maintain.

Promises provide a standard structure for success and failure.

Promise chaining with .then()/.catch() reduces callback nesting.

async/await makes Promise-based code look more like sequential code.

```
// Callback style
getUser(user => {
  getOrders(user, orders => {
    getPayment(orders, payment => {
      sendInvoice(payment, invoice => {
        console.log(invoice);
      });
    });
  });
});
```

```
// Promise style
getUser()
  .then(getOrders)
  .then(getPayment)
  .then(sendInvoice)
  .catch(handleError);
```

11. Important Playground Lesson

If a playground shows only 'Getting student data...' and 'Promise started...' and never reaches the result, check the execution environment.

A real browser or Node.js runtime keeps the event loop alive while setTimeout/API work is pending.

Make sure the function is actually called: displayStudent();

The Promise itself is still correct: pending → resolved/rejected → continuation runs.

```
async function displayStudent() {
    console.log("Getting student data...");

    const student = await getStudentData();

    console.log("Student data received!");
}

displayStudent();
```

Expected: result appears after the simulated 2-second delay

12. Quick Summary

Promise	Represents a future result
resolve()	Success
reject()	Failure
.then()	Handles success
.catch()	Handles errors
.finally()	Runs after settlement
async	Marks a function as asynchronous
await	Gets the Promise result inside async code